

Generador Académico RAG

Un sistema de IA generativa que sintetiza artefactos académicos personalizados anclados en el historial real del grupo y el plan de estudios, mediante un pipeline RAG construido de punta a punta.

GRUPO 14	Martin, Gentil, Natalichio, Cuenca, Ocampo, Aubert
PIPELINE	pdfplumber → MiniLM → Chroma → qwen2.5:14B (Ollama)
ENTORNO	híbrido: pipeline local · generación en GPU (Colab)

01 Planificación y metodología

El objetivo es construir un sistema de IA generativa que, alimentado por una base vectorial con el historial académico del grupo más la documentación de la carrera, **genere artefactos académicos personalizados**: planes de cursada justificados, informes de trayectoria, cartas de pasantía, recomendaciones de orientación, diagnóstico grupal y simulaciones what-if.

El reencuadre central del proyecto fue pasar de un *asistente de preguntas* (que un SQL o NotebookLM resolverían) a un **generador**: cada salida es texto nuevo sintetizado, con razonamiento y estilo, no un campo de una tabla. Un SQL responde qué materias *podés* cursar; este sistema redacta qué *conviene* y por qué, cruzando notas, correlativas y patrón de rendimiento.

La metodología siguió el pipeline de Ciencia de Datos completo: ingesta y preprocesamiento de un corpus mixto, vectorización, análisis exploratorio, visualización, clustering, generación con control de tono y evaluación cuantitativa del anclaje (grounding).

439

documentos en el corpus

36

correlatividades parseadas

6

generadores de artefactos

3

tonos controlables

02 Datos, EDA y preprocesamiento

El corpus combina lo **estructurado** (notas, años y correlatividades) con lo **no estructurado** (chunks del plan de estudios). Las fuentes son PDFs reales: los estados académicos y exports de exámenes de los **seis integrantes del grupo**, y el Plan de Sistemas 2023 (correlatividades del Anexo I y equivalencias del Anexo II).

El preprocesamiento crítico fue el **join entre el estado académico y el plan**: los códigos institucionales no coinciden entre documentos, así que la clave de unión es el *nombre de materia normalizado* (minúsculas, sin tildes ni puntuación). Se parsearon las 36 obligatorias con sus correlativas, resolviendo casos borde como las integradoras, cuyo nombre se parte en dos líneas. El corpus final tiene 439 documentos: 383 de estados académicos, 36 de correlatividades y 20 chunks del plan.

El historial de notas de exámenes usa los nombres del **Plan 2008**, mientras que el estado académico ya migró a los del **Plan 2023**. Para que el cruce no pierda notas aprobadas por equivalencia, incorporamos el **Anexo II** del plan (régimen de equivalencias 2008↔2023) como tabla de canonización de nombres: toda materia se lleva a su nombre 2023 antes del join, con tolerancia a la truncación de nombres largos del PDF.

El export de exámenes resultó además **más completo que el estado académico**: contiene materias aprobadas (con su nota) que ni siquiera figuran como fila en el estado. Esas materias se incorporan al corpus como aprobadas, evitando que el planificador las recomiende como pendientes. En paralelo, se distingue el estado real de cada materia —aprobada, en curso («cursando», aún sin nota) o no iniciada— para que el plan de cursada excluya lo que ya se está cursando y ofrezca, además de las obligatorias habilitadas, las electivas del plan todavía disponibles.

El análisis exploratorio trabaja sobre **220 registros reales** de notas (una entrada por materia aprobada con calificación numérica), correspondientes a los seis integrantes. Los promedios van de **8.0 a 8.7**, un grupo parejo y de buen rendimiento; las diferencias aparecen al desagregar por área temática, que es lo que explota el diagnóstico grupal.

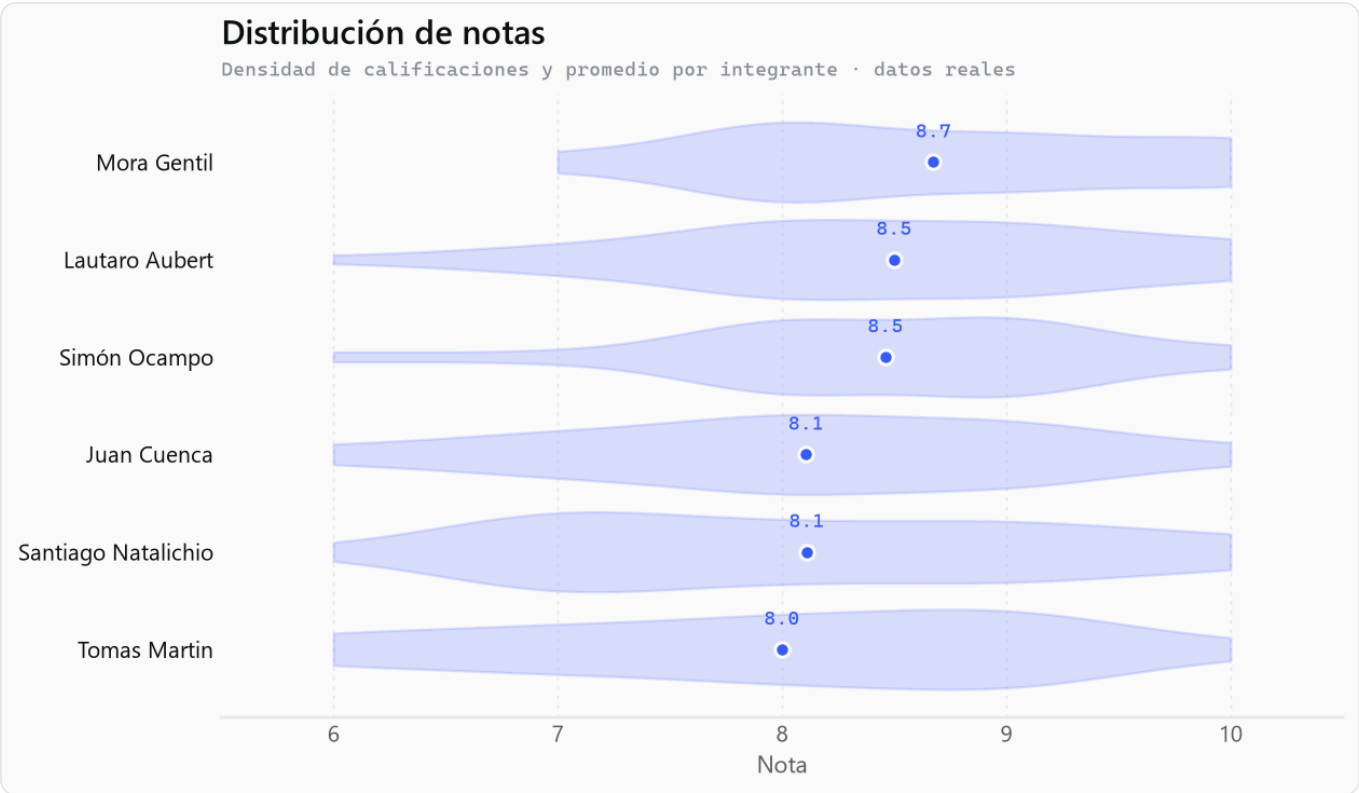


Fig. 1 – Distribución de notas por integrante (datos reales de los seis): promedios entre 7.9 y 8.7.

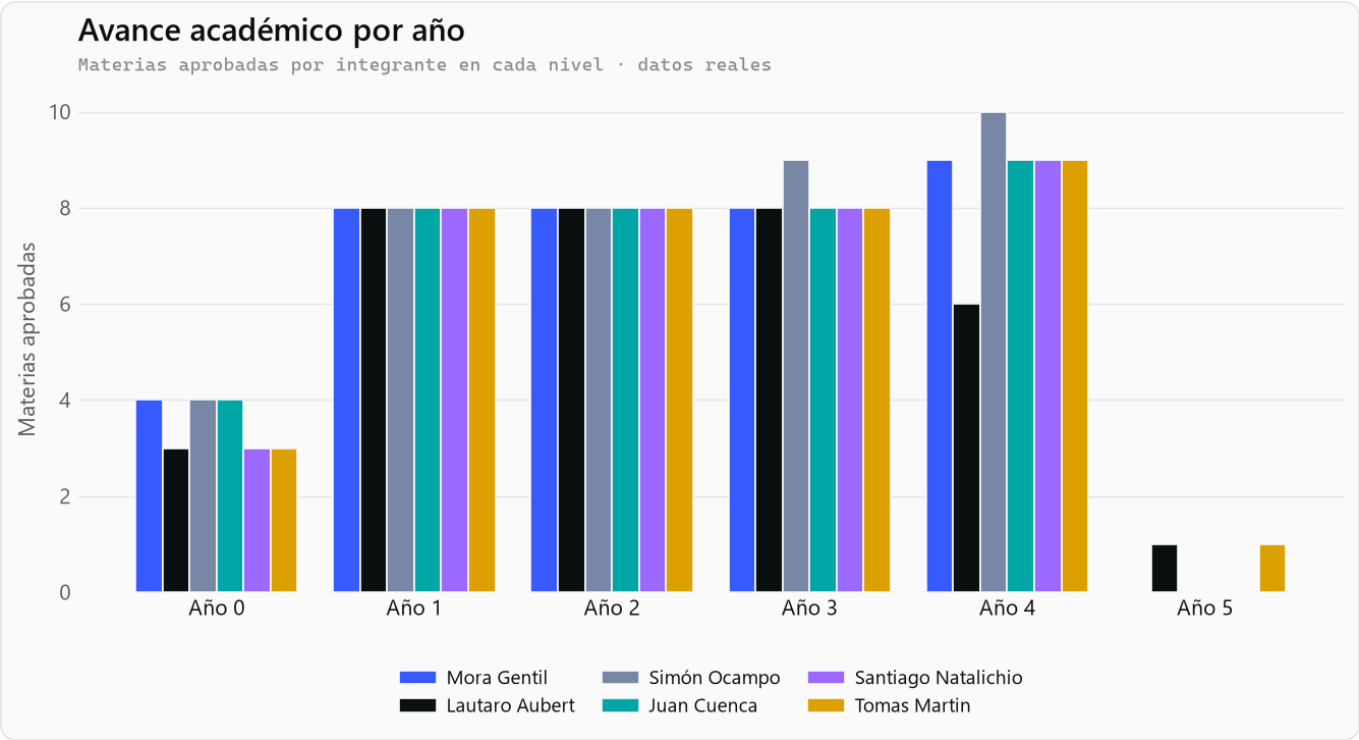


Fig. 2 – Avance académico real por año: materias aprobadas por integrante.

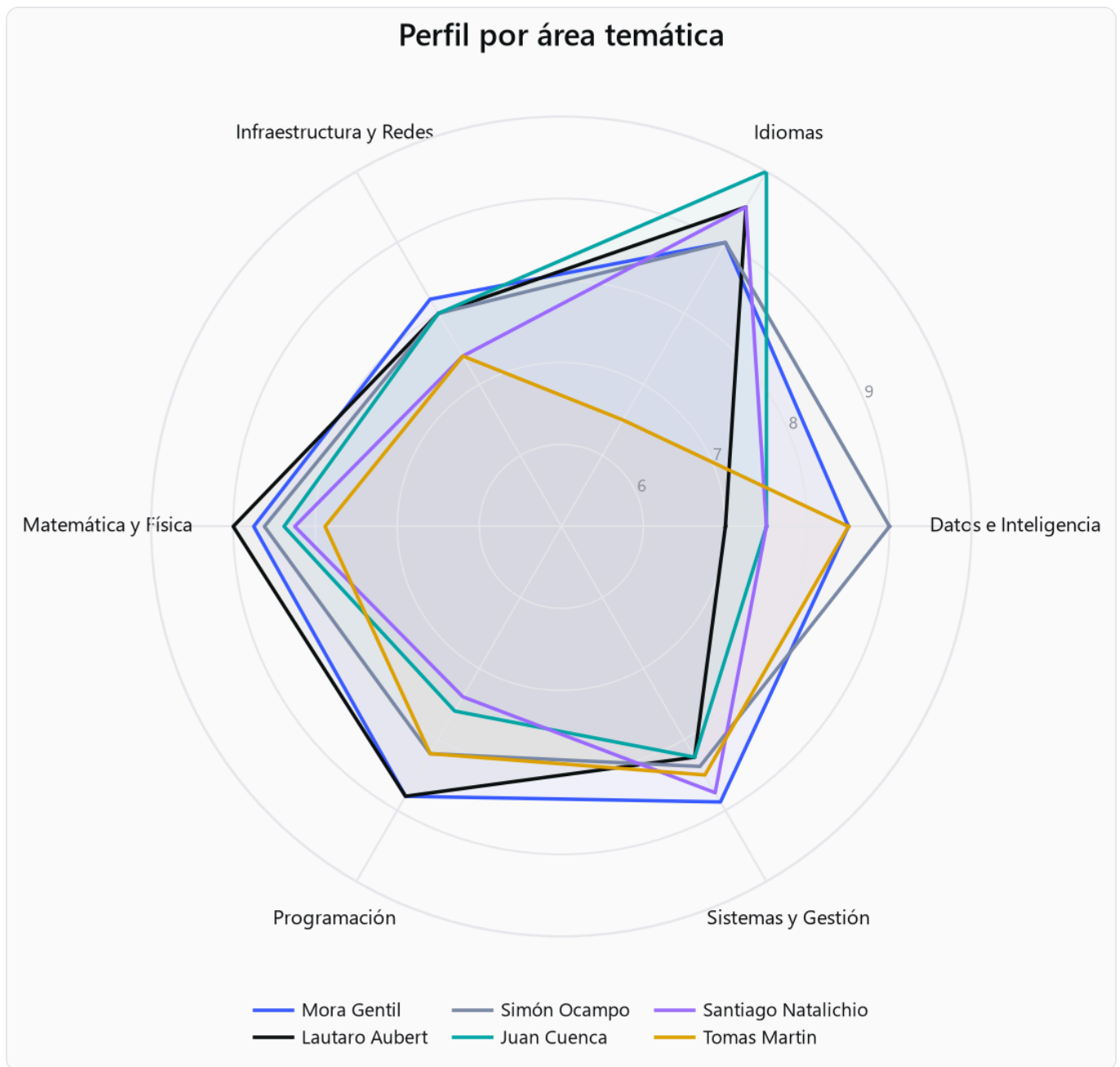


Fig. 3 – Perfil por área temática. Las 36 obligatorias se agrupan en 6 áreas para comparar fortalezas.

03 Pipeline RAG e implementación

El pipeline se construyó en módulos independientes, cada uno con self-checks:

```
extract.py      PDF → texto (pdfplumber)
documents.py    texto → documentos + correlatividades; normalizar() = clave de join
ingest.py       corpus → Chroma; buscar() semántico + obtener() por metadata
rag.py          chat() al LLM (qwen2.5:14B vía Ollama, local o Colab); fallback si está ap
analisis.py     DataFrame de notas reales (6 integrantes) + agregaciones por área
figuras.py      EDA, radar, clustering (KMeans/PCA), t-SNE de embeddings
generar.py      6 generadores de artefactos con control de tono
evaluar.py      grounding léxico + semántico + comparar_con_sin_rag()
```

La vectorización usa **all-MiniLM-L6-v2** (el embedding por defecto de Chroma, local y offline). El retrieval tiene dos modos: **semántico** (similitud de embeddings, para lenguaje natural) y **determinístico** (por metadata, para anclar exactamente las materias aprobadas de un alumno y las correlatividades). La generación corre sobre **qwen2.5:14B vía Ollama**; si Ollama está apagado, la parte RAG sigue funcionando y devuelve el contexto recuperado.

Arquitectura de ejecución híbrida

Todo el pipeline RAG —extracción, embeddings, base vectorial y recuperación— corre **local** en la máquina del grupo. Lo único que cambia de lugar es la **generación del LLM**: como `rag.py` consume Ollama por HTTP (variable OLLAMA_URL), el modelo puede correr en la misma PC (CPU) o en una **GPU T4 gratuita de Google Colab**, expuesta por un túnel de URL fija. Esto permite usar un modelo grande (14B) con respuestas en segundos sin GPU propia, sin cambiar una línea del pipeline. Es una decisión de despliegue: en producción el sistema corre 100% local y offline; para la demo se aprovecha la GPU remota. Los prompts nunca pasan por un proveedor de inferencia de terceros —corren en una VM efímera del propio grupo—, preservando la privacidad de los datos académicos.

Estrategia de prompting

El sistema instruye al modelo a usar **exclusivamente** el contexto recuperado y a no inventar notas, materias ni correlatividades, pero también a **interpretar y seleccionar**: elegir los pocos datos relevantes y explicar qué revelan, en lugar de enumerar todas las notas. Cada artefacto define una tarea concreta y una estructura esperada (por ejemplo, la carta de pasantía exige primera persona, conectar fortalezas con competencias del puesto y no listar el historial). El **control de tono** (técnico, motivacional, honesto) reescribe el mismo dato en distinto registro: es lo que hace al sistema generativo y no un reporte fijo.

"Trabajás solo con los datos del contexto: no inventes ni supongas. Pero no sos un loro de datos: interpretá y seleccioná los datos relevantes y explicá qué revelan; nunca enumeres todas las notas."

04 Clustering y espacio semántico

Sobre el perfil por área (estandarizado) se aplicó **KMeans** y se proyectó a 2D con PCA para visualizar grupos de perfiles afines. Por separado, se proyectaron las 36 obligatorias con **t-SNE sobre sus embeddings**: las materias de la misma área caen juntas, lo que valida que el espacio semántico capturó la estructura del plan, condición necesaria para que el recomendador por matching semántico funcione.

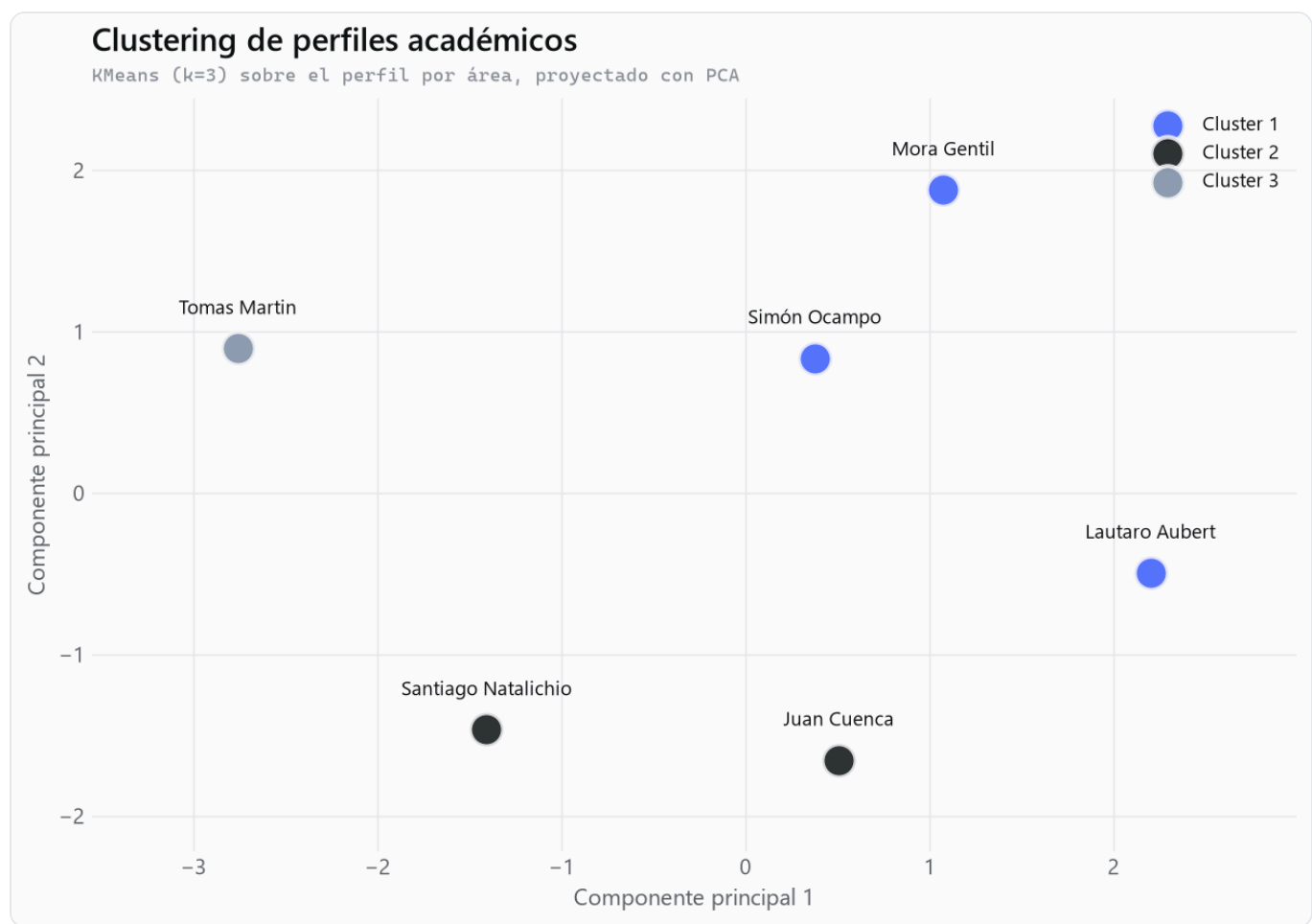


Fig. 4 – Clustering de perfiles académicos (KMeans sobre perfil por área, proyección PCA).

Materias en el espacio semántico

t-SNE de los embeddings de las 36 obligatorias, coloreadas por área

- Datos e Inteligencia
- Idiomas
- Infraestructura y Redes
- Matemática y Física
- Programación
- Sistemas y Gestión



Fig. 5 – Las 36 obligatorias en el espacio semántico (t-SNE de embeddings), coloreadas por área.

05 Evaluación: grounding con/sin RAG

Medimos el anclaje con **cuatro métricas sobre tres ejes**. **(a) Solapamiento:** el **grounding léxico** es la fracción de tokens informativos de la salida (materias, notas) presentes literalmente en el contexto —explicable, pero penaliza la paráfrasis— y el **grounding semántico** promedia el máximo coseno entre cada oración de la salida y los fragmentos del contexto (embeddings MiniLM), reconociendo la reformulación legítima. **(b) Corrección factual:** la **precisión factual** extrae los pares (materia, nota) que la salida afirma y verifica cada uno contra el registro real del alumno —mide acierto, no cercanía—. **(c) Juicio de un LLM:** la **faithfulness** usa el propio modelo como evaluador, puntuando de 0 a 1 cuánto se apoya la respuesta solo en el contexto.

La **demo estrella** compara el mismo pedido **con RAG** (contexto real) contra **sin RAG** (sin contexto). Promediando cuatro artefactos (plan de cursada, recomendación de orientación, informe de trayectoria y carta de pasantía) con **qwen2.5:14B**, el resultado más nítido es la **precisión factual**: con RAG es **1.0** —de cada par (materia, nota) que el modelo afirma, el 100% coincide con el registro real: cero notas inventadas— mientras que sin RAG el modelo no cita ningún dato verificable. La **faithfulness** juzgada por el LLM cae de **0.73 con RAG a 0.50 sin RAG**, y el grounding de **0.55 a 0.36** (léxico) y **0.57 a 0.51** (semántico). En las cuatro métricas, con RAG > sin RAG: la evidencia medible de que el conocimiento privado es el corazón del producto, no un adorno.

Vale una lectura crítica. El grounding léxico **premia la cita literal**: un artefacto profesional que parafrasea e interpreta (como la carta, que por diseño *no* enumera notas) obtiene un léxico más bajo aun siendo correcto —por eso ronda 0.5 y no 1—; es la razón por la que sumamos la precisión factual, que sí mide acierto. Además, como la generación es estocástica, el valor absoluto de una corrida tiene **varianza** (entre corridas el léxico osciló ~0.55–0.64); lo robusto es la **dirección** (con RAG > sin RAG en las cuatro métricas) y la precisión factual de 1.0. Limitación de fondo: ninguna métrica detecta *contradicciones* lógicas sutiles, solo cercanía, corrección de pares y un juicio agregado.

1.0	0.73	0.55	0.57
precisión factual CON RAG (pares materia-nota correctos)	/ 0.50	/ 0.36	/ 0.51
	faithfulness (juez LLM) con / sin RAG	grounding léxico con / sin RAG	grounding semántico con / sin RAG

06 Análisis crítico y conclusiones

El sistema cumple el objetivo: construimos el pipeline RAG completo (no una caja negra) y lo usamos para **generar**, no para responder. La demo con/sin RAG y el grounding hacen visible y medible el valor del conocimiento privado.

Limitaciones

El grounding, aun en su versión semántica, no detecta *contradicciones* sutiles: mide cercanía con el contexto, no consistencia lógica con él; mide densidad de datos citados, no corrección del razonamiento. El recomendador por matching se apoya en nombres de materias y correlativas, no en las descripciones reales de cada cátedra. Y la generación, anclada en el contexto y acotada por el retrieval determinístico por metadata, puede aun así deslizar matices imprecisos que solo una verificación de consistencia detectaría.

Trabajo futuro

Incorporar las descripciones de cátedras y electivas al corpus para que el recomendador compare contenidos y no solo nombres; reforzar la evaluación con métricas de RAG más finas —*faithfulness* y *answer/context relevancy* con un LLM como juez, o precisión de recuperación (hit-rate / MRR) sobre el retriever— que capturen consistencia y no solo cercanía léxica; y consolidar la arquitectura híbrida (modelo grande en GPU para la demo, base local reproducible para producción).

El código, el notebook integrador y la web app de demostración en vivo acompañan este informe como entregables ejecutables.